AI Engineer Interview Case – Contradiction Detection in Tender Documents

Take-home assignment for the AI Engineer technical interview.

Situation

I am a bid manager at an engineering consultancy. When a client invites us to bid for a construction project, they send us a large set of documents (the tender pack) describing what they want built: the client's requirements, technical specifications, drawings, schedules, contract conditions and later corrections. These documents are written by different people at different times, and they often disagree with each other. My job is to find those contradictions before we commit to the project. A single pack can run to 10–30 documents and 500–2,000 pages, so reading everything side by side is slow and error-prone.

Contradictions between those documents cost us money. A kitchen schedule that says 400 kW where the specification says 4 kW, a programme that sums to 14 months against a contract that says 12, or a drawing note that specifies a different fire rating than the specification, all end up as either a mispriced bid or a claim after award. Today we find these by having experienced engineers read everything, and we still miss some.

I want a tool that uses a Large Language Model to read the pack for me, find where the documents contradict each other, and show me exactly where and why, so my team can confirm or dismiss each finding in minutes rather than days.

Functional requirements

Build an application that ingests the pack and produces a list of contradiction findings. A chat interface is optional; a terminal or simple web output is fine.

Each finding must contain:

- The two (or more) conflicting statements, quoted
- Source location for each: document, page or sheet, and section heading, table, or equipment tag where applicable
- The entity or topic the conflict is about (e.g. `Freezer FZ-01 / electrical load`, `Completion date`)

- A category: numeric, unit, scope, date/arithmetic, omission, terminology, superseded text, other
- A severity (high / medium / low) and a confidence score

The application must:

1. Run a full-pack scan and output all findings ranked by severity, presented in a structured form of your choice.
2. Answer targeted questions in natural language, for example:
   - "Are there any conflicts about the electrical load of the kitchen equipment?"
   - "Does the completion date in the contract agree with the programme?"
   - "Do the fire ratings in the door schedule match the architectural specification?"
   - "Has Addendum 2 changed anything that the M&E spec still contradicts?"
   - "What does the drawing say about the ceiling height in the main hall, and does the Employer's Requirements agree?"
3. Support follow-up questions on a finding ("Where else is this value mentioned?", "Show me the full clause").

Requirements (choose only one approach)

- The application should be implemented as a simple web application packaged as a Docker image.
- Alternatively, you may demonstrate a simple model within the terminal or console of your IDE.

Data

A tender pack for a fictional project, *Fjordview Community Centre*, has been prepared for this purpose. It consists of seven PDF documents of roughly 10 pages each:

1. `01_Instructions_to_Tenderers.pdf` – submission rules, definitions, order of precedence, key dates
2. `02_Particular_Conditions_of_Contract.pdf` – completion, damages, insurance, payment
3. `03_Employers_Requirements.pdf` – scope, performance criteria, room data sheets
4. `04_Architectural_Specification.pdf` – materials, fire ratings, door and partition schedules

5. `05_MEP_Specification.pdf` – electrical, mechanical, ventilation, equipment schedules
6. `06_Drawing_Notes_and_Tender_Programme.pdf` – drawing register, drawing notes, programme
7. `07_Addenda_and_Tender_Query_Responses.pdf` – addenda and the tender Q&A log

Important: this is a demonstration dataset. It is deliberately small so that it can be run on a laptop, and it contains a number of planted contradictions as well as statements that look contradictory but are not. Real tender packs are 20–50 times larger. Your solution should be designed to solve the problem in general, for any tender pack, rather than tuned to this specific dataset. We will ask you about how your approach scales and generalises.

Deployment and preparations

Build the application before the interview and be prepared to demonstrate it during the interview.

- If you have access to a cloud environment (Azure, AWS, GCP), you can deploy it there. Otherwise, run it on your local machine.
- To run the model, you can use your own LLM API key. If you do not have one, please send an email to Rohan.KumarYadav@aecom.com and we will provide a temporary API key valid for 48 hours.
- Please include a short README on how to run the application and a short note (half a page) on your design: how you decide what to compare with what, and how you keep heading, table and entity context.

During the interview

Be prepared to:

- Demonstrate a full-pack scan and walk us through all the contradiction findings.
- Run the targeted questions above live.
- Discuss: chunking vs structured extraction; entity-centric vs pairwise comparison; LLM-as-judge reliability and calibration; evaluation without ground truth; confidentiality of tender data and on-premise or private deployment; how the design changes for a 2,000-page pack.
- Answer general questions about LLMs, RAG, agents and the AI space.

If you could not implement everything, that is expected. We want to understand how you think.

You can use whichever tools you feel most comfortable with.

FYI: Please share your solution code with us as a GitHub repository link.